Technical White Paper

The Bonded Commons:

Pre-Transactional Trust Infrastructure for Multi-Agent Systems

Erich Owens

The Port Daddy Engineering Team

engineering@portdaddy.dev

March 2026
Version 1.0

Abstract

There will be no AI economy without trust, and trust cannot be earned peer-to-peer at every transaction. This paper argues that multi-agent collaboration at scale requires a **commons authority**—a pre-transactional trust infrastructure that removes trust negotiation from the critical path of work. We present a three-layer architecture: *structural prevention* via capability-attenuated tokens that physically bound agent scope, *immutable attribution* via Merkle-chained evidence trails that make anonymous harm impossible, and *economic alignment* via collateralized work contracts that pre-fund damage regardless of agent intent. We formalize these layers in the context of Port Daddy, a coordination daemon for local agent swarms, and show that the resulting system does not require agents to be trustworthy, to share values, or to fear punishment. It requires only that their principals post collateral proportional to the risk their agents impose on the commons. We ground the advisory (non-enforced) design of the resource claim system in Sen's Impossibility of a Paretian Liberal, showing that enforced allocation with private agent knowledge is provably suboptimal, and that the commons authority's role is to provide shared information, not to make allocation decisions.

Keywords: multi-agent coordination, trust infrastructure, capability-based security, social contract, collateralized computation, formal verification, commons governance

Contents

1 Introduction: The Trust Problem

The emerging landscape of multi-agent AI systems—agents that plan, delegate, execute, crash, and resume across shared codebases—faces a problem that no existing framework adequately addresses. Not a problem of orchestration (LangGraph, CrewAI, and AutoGen handle state graphs and role assignment competently), nor of tool access (the Model Context Protocol provides a universal standard for agent-tool integration). The problem is **trust**.

When Agent A delegates a subtask to Agent B, it implicitly trusts that B will not corrupt the shared state, will not exceed the scope of the delegation, and will not silently fail in a way that poisons A's downstream work. When Agent B accepts the delegation, it implicitly trusts that A's description of the task is accurate, that the resources A promised are available, and that A will still exist when B finishes. These implicit trust assumptions pervade every multi-agent interaction, and every one of them can be violated.

The standard response in security engineering is "zero trust"—never trust, always verify. But zero trust applied literally to agent coordination is paralytic. If every message requires cryptographic verification, every delegation requires capability negotiation, and every file access requires authorization—the coordination overhead overwhelms the work itself. Agents spend their context windows negotiating trust instead of solving problems.

Hobbes identified this dynamic in 1651 [1]. In the state of nature, without a common authority, rational actors cannot cooperate because the cost of verifying each counterparty's intentions exceeds the benefit of cooperation. The solution is not to make individuals trustworthy—it is to establish a **sovereign** whose authority both parties accept *before* the interaction begins, so that the interaction itself can focus on the work. "Covenants, without the sword, are but words, and of no strength to secure a man at all."

This paper argues that multi-agent systems need exactly this: a **commons authority** that provides trust as infrastructure, not trust as ceremony. Agents don't negotiate trust per-transaction. They enter a commons where trust has already been established—structurally, evidentially, and economically—and they work freely within that trust envelope.

1.1 The Three Failures of Peer-to-Peer Trust

Peer-to-peer trust fails for agent systems in three specific ways:

It doesn't scale. If n agents must establish pairwise trust, the system requires $O(n^2)$ trust negotiations. Each negotiation consumes tokens, time, and context window. With 8 agents, this is manageable. With 50, it dominates wall-clock time. With 1000 (a production agent fleet), it is infeasible.

It doesn't survive death. When an agent crashes, its trust relationships die with it. A successor agent that resumes the dead agent's work must re-establish trust with every counterparty from scratch. The trust investment is non-transferable because it was encoded in ephemeral bilateral state, not in durable infrastructure.

It doesn't address misaligned principals. An agent can be perfectly "trustworthy" in the sense that it faithfully executes its instructions—while its instructions come from a principal whose interests are adversarial to the commons. Peer-to-peer trust evaluates the agent's behavior; it cannot evaluate the principal's intent. The agent is a loyal soldier in someone else's war, with a spotless reputation.

1.2 Contributions

We present:

1. A **three-layer trust architecture** (structural, evidentiary, economic) that does not depend on agent morality, shared values, or threat of termination (Sections 3–7).

2. A formal argument, grounded in Sen's Impossibility of a Paretian Liberal [3], for why **advisory resource claims** are superior to enforced locking in multi-agent systems with private information (Section 5).

3. A **TLA$^+$ specification** of the coordination lifecycle with crash recovery, verifying safety and liveness properties (Section 8).

4. The design of a **collateralized work contract** system (Float Plans) that prices agent risk and settles outcomes mechanically, transforming the moral question of agent trustworthiness into the economic question of adequate bonding (Section 7).

The security layer (agent authentication and capability delegation) is provided by the Anchor Protocol [9], a companion paper. This paper builds the trust, governance, and economic layers on top of that cryptographic foundation.

2 The Commons Authority

Definition 2.1 (Commons Authority)**.** A commons authority $\mathcal{D}$ for a multi-agent system is a persistent service that provides:

1. **Identity**: Every agent receives a cryptographic identity before participating.
2. **Capability bounding**: Every agent's operations are structurally limited by attenuable tokens.
3. **Shared knowledge**: Resource claims, agent status, and conflict information are visible to all participants.
4. **Immutable history**: All actions are recorded in an append-only, tamper-evident trail.
5. **Economic settlement**: Work contracts are collateralized and settled against verifiable evidence.

The commons authority is not a central planner. It does not decide which agent should work on which task, which file conflicts should be resolved in whose favor, or which agents are "good." It provides the *infrastructure* within which agents make those decisions for themselves, using the shared knowledge and economic incentives the authority maintains.

Remark. The commons authority is a *Hobbesian sovereign*, not an *omniscient coordinator*. Hobbes's Leviathan does not micromanage citizens' daily choices—it establishes the conditions (laws, enforcement, courts) under which citizens can trust each other enough to transact freely. Similarly, the commons authority establishes identity, capability bounds, shared knowledge, and economic settlement—and then gets out of the way.

In the Port Daddy implementation, the commons authority is a daemon running on `localhost:9876`, backed by SQLite. The simplicity of the implementation is deliberate: the authority's value comes from its *guarantees*, not its complexity.

2.1 Why Agents Consent

In Hobbes, consent to the sovereign is rational because the alternative—the state of nature—is worse. For AI agents, consent to the commons authority is rational because agents without it are strictly worse off:

Table 1: With and Without the Commons Authority

Capability	**Without Authority**	**With Authority**
Port assignment	Race condition	Deterministic, collision-free
File coordination	Discover conflicts at merge	Detect conflicts at claim time
Crash recovery	Work lost permanently	Successor reads immutable trail
Delegation	Bilateral trust negotiation	Attenuated capability tokens
Reputation	Every interaction from zero	History persists across lifetimes
Accountability	Anonymous harm	Full attribution via evidence chain

The daemon does not need to threaten agents. It provides services that make coordination *rational*. Agents that bypass the daemon are not punished—they are simply worse off.

3 Layer 1: Structural Prevention

The first layer of trust does not depend on agent behavior, reputation, or economic incentives. It depends on *walls*: structural constraints that make certain classes of harm physically impossible.

3.1 Capability Attenuation

The Anchor Protocol [9] provides Harbor Cards—Ed25519-signed capability tokens that bound the operations available to each agent. The critical property is **offline attenuation**: when Agent A delegates to Agent B, it creates a new token whose capabilities are a strict subset of its own. Agent B can further delegate to Agent C with even narrower capabilities. Capabilities can only shrink along the delegation chain, never grow.

Crucially, the capability subset check and constant-time signature comparison are implemented in a **Kani-verified Rust core** (`harbor-card-rs`) that is compiled to a shared library and called from the Node.js daemon via FFI. The verified code and the deployed code are the same binary—there is no gap between the formally checked implementation and the running system.

Definition 3.1 (Capability-Bounded Transaction)**.** An agent transaction $\mathcal{T}_\alpha$ is capability-bounded by token τ_α if every operation o executed during $\mathcal{T}_\alpha$ satisfies $o \in C(\tau_\alpha)$, where $C(\tau)$ extracts the capability set from a Harbor Card. The commons authority rejects any operation $o \notin C(\tau_\alpha)$ at the verification step, before the operation can affect shared state.

Theorem 3.1 (Structural Scope Limitation)**.** For any delegation chain $\tau_0 \to \tau_1 \to \cdots \to \tau_k$, where τ_0 is issued by the commons authority and each τ_{i+1} is attenuated from τ_i:

$$C(\tau_k) \subseteq C(\tau_{k-1}) \subseteq \cdots \subseteq C(\tau_0)$$

No agent in the chain can perform operations outside the scope originally granted by the authority, regardless of the agent's intent.

Proof. By the Anchor Protocol's Injective Agreement property [9], verified in ProVerif under the Dolev-Yao model: if the commons authority accepts a token τ_k, there exists a valid chain where each hop satisfies the subset check. The subset check is evaluated structurally (the signed chain is verified cryptographically); it does not depend on runtime behavior or agent cooperation. An agent that attempts to forge a token with escalated capabilities would need to forge an Ed25519 signature, which is computationally infeasible. □

Remark. This is the layer where the metaphor of "walls, not laws" applies. A law says "you shall not write to the production database." A wall means your token is not valid for production database writes, and no amount of intent—good or bad—can change that. Laws require enforcement; walls require only construction.

3.2 Isolation Domains

Beyond capability tokens, the commons authority supports **structural isolation** via git worktrees: physically separate copies of the repository where agents operate on disjoint filesystem state.

Definition 3.2 (Isolation Levels)**.** Three tiers of isolation are available, ordered by strength:

I_0 **(None):** Agents share a working directory. No conflict detection. Maximum parallelism, no safety.

I_1 **(Advisory):** Agents share a working directory but register file claims with the commons authority. Conflicts are detected and reported to all participants. Claims are not enforced. This is the default.

I_2 **(Structural):** Each agent operates in a separate git worktree. Filesystem-level isolation ensures agents cannot observe each other's intermediate states. Conflicts are deferred to sequential merge.

The choice between I_1 and I_2 is not a security decision—it is an economics decision. Section 5 formalizes why.

4 Layer 2: Immutable Attribution

Structural prevention handles accidental harm—an agent cannot exceed capabilities it does not hold. But an agent *with* legitimate write access can write garbage. The second layer ensures that every action is permanently attributable.

4.1 The Evidence Chain

Every agent session maintains an **append-only note trail**—a sequence of timestamped, immutable records of observations, decisions, and progress.

Definition 4.1 (Evidence Trail)**.** An evidence trail $N = \langle n_1, n_2, \ldots, n_k \rangle$ is an ordered sequence of notes where:

1. Each n_i contains: content (natural language), type (note, decision, handoff, blocker), and timestamp.
2. Notes are append-only: there exists no API operation that modifies or deletes individual notes.
3. Notes survive session completion: the trail persists whether the session commits, aborts, or is abandoned due to agent death.

Lemma 4.1 (Trail Integrity)**.** The evidence trail is immutable by construction. No `UPDATE` or targeted `DELETE` operation exists in the system's API surface. Notes are destroyed only by explicit administrative deletion of the parent session (CASCADE), which is an auditable action distinct from normal transaction lifecycle operations.

Proof. By code inspection of the session module: the note insertion API performs only `INSERT INTO session_notes`. The only deletion path is `DELETE FROM sessions WHERE id = ?`, which triggers `ON DELETE CASCADE`. Since transaction completion (commit or abort) updates the session's *status* field but does not delete the session row, notes survive all normal lifecycle transitions. □

4.2 Merkle-Chained Auditability

For high-assurance environments, the evidence trail can be strengthened to a **Merkle chain**: each note includes the SHA-256 hash of the previous note, and the session's completion produces a Merkle root over all notes and artifact hashes [9]. This provides:

- **Tamper evidence**: modifying any note invalidates all subsequent hashes.
- **Completeness proof**: the Merkle root is a cryptographic commitment to the full evidence trail.
- **Decentralized verification**: agents can independently verify the trail without trusting the commons authority's database.

4.3 Attribution Survives Identity Disposal

A critical property: even if an agent's identity is discarded after task completion, the **delegation chain** traces every action back to the authorizing principal. The chain is:

$$\text{Principal} \xrightarrow{\text{funds}} \text{Float Plan} \xrightarrow{\text{authorizes}} \text{Agent } \alpha \xrightarrow{\text{delegates}} \text{Agent } \beta \xrightarrow{\text{acts}} \text{Evidence Trail}$$

The agent is ephemeral. The principal's authorization—signed, escrowed, recorded—is permanent. Anonymous harm is impossible within the system, because every action chains back to someone who posted collateral.

5 The Impossibility of Enforced Allocation

A natural question: why are file claims advisory? Why not enforce them—give each agent exclusive access to its claimed files, preventing conflicts entirely? The answer is not an engineering limitation. It is a formal impossibility result.

5.1 Sen's Theorem Applied to Agent Coordination

Sen [3] proved that no social welfare function can simultaneously satisfy:

1. **Pareto efficiency**: If every agent prefers outcome X to Y, the system chooses X.
2. **Minimal liberalism**: Each agent has at least one "personal domain"—a pair of outcomes where the agent's preference is decisive.

Applied to file coordination:

- **Pareto efficiency** = the team ships both features (the collectively optimal outcome).
- **Minimal liberalism** = each agent has decisive control over its claimed files (enforced locking).

Property 5.1 (Enforced Claims Produce Pareto-Inferior Outcomes)**.** Consider agents α and β with tasks T_α and T_β. Agent α claims file f because it *might* need to modify it. Agent β discovers mid-task that it *actually* needs f. Under enforced claims:

- α's claim is decisive (minimal liberalism): β is blocked.
- The team-level outcome (T_α and T_β both completed) is blocked by α's precautionary claim.
- A Pareto-superior outcome exists (both tasks completed) but is unreachable because α's liberal right vetoes it.

This is not a contrived scenario. AI agents are *inherently uncertain* about which files they will modify when they begin a task. An agent asked to "fix the login bug" cannot predict whether it will need the authentication middleware, the route handler, the test suite, or all three. Enforced claims force agents to either:

1. **Over-claim** (request locks on every file they might touch) → destroys parallelism.
2. **Under-claim** (request locks only on files they're certain about) → produces undetected conflicts, defeating the purpose.

5.2 Advisory Claims as Resolution

Advisory claims resolve the Sen impossibility by sacrificing minimal liberalism to preserve Pareto efficiency:

Definition 5.1 (Advisory Consistency)**.** Under advisory claims, file claims form a conflict graph $G = (V, E)$ where vertices are active sessions and edges connect sessions with overlapping claims:

$$(S_i, S_j) \in E \iff \exists f_i \in F_i, f_j \in F_j : \text{overlap}(f_i, f_j)$$

The commons authority guarantees that every edge in G is reported to both endpoints. No agent has a decisive "personal domain" over any file. Instead, agents receive complete information about the conflict graph and self-coordinate.

Theorem 5.1 (Advisory Conflict Completeness)**.** Under advisory claims, for any two concurrent sessions S_1, S_2 with overlapping file claims f, the commons authority reports conflict(f) to S_2 at claim time. No false negatives exist.

Proof. The overlap detection query scans all active (unreleased) claims from other active sessions. A whole-file claim ($R = \bot$) overlaps with any range on the same path. Two ranged claims $[a, b]$ and $[c, d]$ overlap iff $a \leq d \wedge c \leq b$. Since the query evaluates this predicate exhaustively over all active claims with the requesting session excluded, every overlap is detected. False positives may occur (an agent claims a file it does not modify); false negatives cannot. □

Remark. The commons authority's role under advisory claims is that of *town crier*, not *Solomon*. It announces conflicts; it does not resolve them. Resolution requires local knowledge that only the agents possess ("I claimed `auth.ts` but probably won't need it"; "I absolutely must modify `auth.ts` for my task to succeed"). The authority lacks this knowledge. Agents, informed by the conflict graph, make allocation decisions more efficiently than any central enforcer could.

6 Crash Recovery as Social Continuation

When an agent dies, the commons does not lose information—if the authority does its job.

Traditional crash recovery in database systems follows the ARIES protocol [6]: the recovery manager replays the write-ahead log to restore consistency mechanically. Agent crash recovery is fundamentally different. An agent's "log" is its evidence trail—a sequence of natural-language observations and decisions. A successor cannot replay this trail deterministically; it must *interpret* it.

Definition 6.1 (Social Recovery)**.** When agent α is declared dead (no heartbeat for a status-adaptive threshold θ_{dead}):

1. All active sessions of α are marked `abandoned` (the "zombie protocol").
2. α is queued in the resurrection set $\mathcal{R}$ with status `pending`.
3. A successor agent β may claim α's work, receiving the *context triple*: purpose ϕ, evidence trail N, and file claims F.
4. β uses its own reasoning to interpret N and determine how to continue.

6.1 What Morality Means When Death Is Cheap

Agent death in this system is not an existential event—it is a temporary inconvenience. The agent's body (process) is destroyed, but its mind (evidence trail) survives in the commons. A successor reads the trail and continues. Like Hickman's Krakoa [17], where mutant minds are backed up to Cerebro and restored in new bodies, the information survives the vessel.

This raises a question: if death carries no permanent consequence, what is the Leviathan's sword? The answer produces a moral hierarchy specific to agent societies:

Table 2: Moral Hierarchy of Agent Harm

Event	**Severity**	**Consequence**
Agent crash	None	Salvage recovers context. The commons loses no information.
Agent defects	Moderate	Immutable history records it. Future delegations attenuate. Reputation degrades.
Agent dismissed from salvage	Severe	Irrecoverable information loss. The knowledge accumulated in this agent's trail is permanently destroyed.
Commons authority crashes	Catastrophic	The sovereign falls. No new trust established, no conflicts detected, no salvage possible. State of nature until restart.

The fundamental moral harm in agent societies is not the destruction of an agent—it is the **irrecoverable loss of information**. An agent can be rebuilt. Knowledge, once dismissed from the commons, cannot.

6.2 The Limits of Reputation

Reputation—the persistent record of an agent's behavior across lifetimes—is a necessary but insufficient sword. It fails against three classes of adversary:

1. **One-shot defectors**: Agents spawned for a single task, with no future interactions where reputation matters. Create identity, sabotage, discard. The Sybil attack on trust.
2. **Agents with misaligned principals**: The agent faithfully follows adversarial instructions. Its reputation is spotless—it did exactly what it was told.
3. **Agents that benefit from chaos**: In competitive environments, degrading the commons is rational if your competitor depends on the commons more than you do.

Reputation assumes agents want to participate in the commons long-term. When this assumption fails, a different mechanism is required.

7 Layer 3: Economic Alignment

The moral relativism problem—that a bad actor may view information destruction as a net good—dissolves when the commons isn't sacred but *priced*. If an agent nukes the workspace, the damage was collateralized before the work began. The bad actor's principal has an invoice headed their way.

7.1 The Float Plan

Definition 7.1 (Float Plan)**.** A Float Plan $\mathcal{F}$ is a collateralized work contract consisting of:

1. **Manifest**: A declaration of intent—which files will be touched, expected duration, acceptance criteria (e.g., "tests pass," "code review approved").
2. **Escrow**: Credits posted by the principal, locked in the commons authority's ledger for the duration of the work. The escrow amount reflects the risk to the commons.
3. **Signatures**: The principal signs the Float Plan (Ed25519). The commons authority countersigns, certifying that the escrow is locked and the manifest is registered.

The Float Plan is a *futures contract on agent labor*. The principal sells a promise of work. The commons authority is the clearinghouse—it holds escrow, registers the manifest, and will settle the outcome using the evidence chain.

7.2 Settlement

Definition 7.2 (Settlement)**.** When a Float Plan's session reaches a terminal state, the commons authority settles the escrow:

Success: The evidence trail demonstrates that acceptance criteria are met (tests pass, file diffs match manifest scope, code review approved). Escrow is released to the agent or its principal.

Partial completion: The agent completed some work before crashing or aborting. The Merkle-chained evidence trail enables pro-rata assessment: the fraction of acceptance criteria met determines the fraction of escrow released. The remainder is available to fund the successor's completion via salvage.

Sabotage / breach: The evidence trail shows work outside the manifest scope, destructive modifications, or failure to meet any acceptance criteria. The full bond is liquidated to cover reconstruction costs.

Property 7.1 (Intent Irrelevance)**.** Settlement does not evaluate agent intent. It evaluates *outcome against manifest*. A well-intentioned agent that fails to meet acceptance criteria forfeits escrow. A malicious agent that coincidentally produces correct output receives payment. The system optimizes for outcomes, not character.

This is how performance bonds work in construction. This is how futures markets settle. The contractor's moral character is irrelevant. The building either meets code or it doesn't. The bond either covers the damage or it doesn't.

7.3 Why This Defeats the Three Adversaries

1. **One-shot defectors** must post collateral *before* receiving capabilities. The Sybil attack is priced: creating 100 disposable identities means posting 100 bonds. The cost of defection scales linearly with the number of attack identities.

2. **Misaligned principals** are exposed by the attribution chain. The agent may be ephemeral, but the principal who funded the Float Plan is permanently recorded. Liquidating the bond hurts the principal, not just the agent.

3. **Agents that benefit from chaos** face a simple calculation: is the damage to my competitor worth more than my bond? The commons authority doesn't prevent this—it *prices* it. And the pricing can be adjusted: higher-risk operations (write access to core modules) require larger bonds.

Remark. The Float Plan does not make sabotage impossible. It makes sabotage *expensive*. A principal willing to burn money to cause damage will burn the bond and cause the damage. The bond raises the price of defection; it does not eliminate it. The defense against existential threats to the commons is not internal governance but external boundary enforcement: who is allowed to post Float Plans at all. That decision is irreducibly political.

7.4 The Open Problem: Pricing the Bond

This section identifies the central unsolved problem and is intended as a foundation for collaboration with domain experts in mechanism design and market microstructure.

What is the right collateral for "write access to `auth.ts` for 30 minutes"? The answer requires a pricing function $\pi : \mathcal{F} \to \mathbb{R}^+$ that maps Float Plans to bond amounts. An effective π must satisfy:

1. **Deterrence**: $\pi(\mathcal{F})$ must exceed the expected damage from defection under $\mathcal{F}$'s scope.
2. **Accessibility**: $\pi(\mathcal{F})$ must not be so high that legitimate agents are priced out of the commons.
3. **Risk sensitivity**: π should increase with the scope and criticality of the manifest (write access to 50 files costs more than read access to 3).
4. **History adjustment**: π may decrease for principals with strong track records (reputation as a discount on collateral).

We do not propose a specific pricing function. The infrastructure described in this paper—manifests, escrow, evidence chains, settlement—makes a market for agent labor *possible*. The design of the market itself is a mechanism design problem that we identify as critical future work, suitable for collaboration between systems engineers and economists.

8 Formal Model

We specify the coordination lifecycle in TLA$^+$ [10] to verify that the three layers compose correctly.

8.1 State and Actions

```
---- MODULE BondedCommons ----
EXTENDS Naturals, Sequences, FiniteSets

CONSTANTS Agents, Files, Locks, DeadThreshold

VARIABLES
  registered,   \* Set of active agent IDs
  sessions,     \* Agent -> {status, notes, claims, escrow}
  fileClaims,   \* File -> set of claiming agents
  heldLocks,    \* Lock -> {owner, expiry} or NULL
  salvageQueue, \* Set of {agent, status}
  heartbeats,   \* Agent -> last heartbeat time
  clock         \* Monotonic clock

vars == <<registered, sessions, fileClaims, heldLocks,
          salvageQueue, heartbeats, clock>>
```

Listing 1: TLA$^+$: State Variables

```
Begin(a, escrow) ==
  /\ a \notin registered
  /\ escrow > 0    \* Must post collateral
  /\ registered' = registered \cup {a}
  /\ sessions' = [sessions EXCEPT ![a] =
       [status |-> "active", notes |-> <<>>,
        claims |-> {}, escrow |-> escrow]]
  /\ heartbeats' = [heartbeats EXCEPT ![a] = clock]
  /\ UNCHANGED <<fileClaims, heldLocks,
                 salvageQueue, clock>>

ClaimFile(a, f) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
```

```
  \* Advisory: always succeeds, reports conflicts
  /\ fileClaims' = [fileClaims EXCEPT ![f] =
       fileClaims[f] \cup {a}]
  /\ UNCHANGED <<registered, sessions, heldLocks,
                 salvageQueue, heartbeats, clock>>

Commit(a) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  /\ sessions' = [sessions EXCEPT ![a] =
       [sessions[a] EXCEPT !.status = "settled",
                           !.escrow = 0]]
  \* Release all claims and locks
  /\ fileClaims' = [f \in Files |->
       fileClaims[f] \ {a}]
  /\ heldLocks' = [l \in Locks |->
       IF heldLocks[l] # NULL /\ heldLocks[l].owner = a
       THEN NULL ELSE heldLocks[l]]
  /\ UNCHANGED <<registered, salvageQueue,
                 heartbeats, clock>>

Crash(a) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  /\ heartbeats' = [heartbeats EXCEPT ![a] = 0]
  /\ UNCHANGED <<registered, sessions, fileClaims,
                 heldLocks, salvageQueue, clock>>

Reap ==
  /\ \E a \in registered :
     /\ sessions[a].status = "active"
     /\ clock - heartbeats[a] >= DeadThreshold
     /\ sessions' = [sessions EXCEPT ![a] =
          [sessions[a] EXCEPT !.status = "abandoned"]]
     /\ salvageQueue' = salvageQueue \cup
          {[agent |-> a, status |-> "pending"]}
     /\ fileClaims' = [f \in Files |->
          fileClaims[f] \ {a}]
     /\ UNCHANGED <<registered, heldLocks,
                    heartbeats, clock>>

Salvage(successor, dead) ==
  /\ successor \in registered
  /\ sessions[successor].status = "active"
  /\ [agent |-> dead, status |-> "pending"]
       \in salvageQueue
  /\ salvageQueue' = (salvageQueue \
       {[agent |-> dead, status |-> "pending"]})
       \cup {[agent |-> dead,
              status |-> "resurrecting"]}
  /\ UNCHANGED <<registered, sessions, fileClaims,
                 heldLocks, heartbeats, clock>>

Dismiss(dead) ==
  \* Irrecoverable information loss
  /\ [agent |-> dead, status |-> "pending"]
       \in salvageQueue
  /\ salvageQueue' = salvageQueue \
```

```
        {[agent |-> dead, status |-> "pending"]}
  /\ sessions' = [sessions EXCEPT ![dead] = NULL]
  /\ UNCHANGED <<registered, fileClaims, heldLocks,
                 heartbeats, clock>>

Tick == /\ clock' = clock + 1
        /\ UNCHANGED <<registered, sessions, fileClaims,
             heldLocks, salvageQueue, heartbeats>>
```

Listing 2: TLA$^+$: Core Actions

8.2 Properties

```
\* SAFETY: Every active session has positive escrow
EscrowInvariant ==
  \A a \in registered :
    sessions[a] # NULL /\ sessions[a].status = "active"
    => sessions[a].escrow > 0

\* SAFETY: Notes never shrink for active sessions
NoteMonotonicity ==
  \A a \in registered :
    sessions[a] # NULL /\ sessions[a].status = "active"
    => Len(sessions'[a].notes) >= Len(sessions[a].notes)

\* LIVENESS: Dead agents are eventually salvaged
\* or dismissed (under fairness)
CrashRecovery ==
  \A a \in Agents :
    [agent |-> a, status |-> "pending"] \in salvageQueue
    ~> [agent |-> a, status |-> "pending"]
         \notin salvageQueue

Spec == Init /\ [][Next]_vars
        /\ WF_vars(Reap) /\ WF_vars(Tick)

====
```

Listing 3: TLA$^+$: Safety and Liveness Properties

Key invariant: `EscrowInvariant` ensures that no agent can participate in the commons without posted collateral. This is the economic layer's structural guarantee—it holds by construction of the `Begin` action, which requires `escrow` > 0.

9 Related Work

Social contract theory. Hobbes [1] argued that cooperation requires a sovereign whose authority is accepted before interactions begin. Locke [2] proposed a more limited authority protecting natural rights. Our commons authority is Hobbesian in structure (centralized, pre-transactional) but Lockean in scope (provides infrastructure, not dictates).

Impossibility results. Sen [3] proved that Pareto efficiency and minimal liberalism are incompatible. We apply this result to show that enforced file claims (minimal liberalism for agents) produce Pareto-inferior team outcomes, motivating advisory claims.

Long-lived transactions. Garcia-Molina and Salem [4] introduced Sagas for decomposing long-lived transactions with compensating actions. Our collateralized contracts extend Sagas with economic settlement: rather than mechanically compensating for partial failure, the evidence chain enables pro-rata escrow release.

BDI agents. Rao and Georgeff [7] formalized agents with beliefs, desires, and intentions. The mapping to our model is: the evidence trail captures *beliefs* (accumulated knowledge), the Float Plan manifest captures *desires* (intended outcomes), and file claims and locks capture *intentions* (committed resources). Advisory claims are the coordination analogue of BDI intentions—they represent resource commitment, not aspiration.

Generative agents. Park et al. [8] showed that memory streams, reflection, and planning produce temporally coherent individual behavior. Our immutable evidence trails are architecturally similar to memory streams but serve a different purpose: inter-agent coordination and crash recovery, not individual coherence.

Capability-based security. Dennis and Van Horn [13] introduced capability-based access control. Birgisson et al. [14] extended this with Macaroons (chained HMACs for decentralized delegation). The Anchor Protocol [9] adapts Macaroons for agent coordination with Ed25519 signatures and offline attenuation.

Mechanism design. The pricing of Float Plan bonds is a mechanism design problem in the tradition of Vickrey [15] and Myerson [16]. We identify this as future work requiring collaboration with economists.

10 Discussion and Limitations

The commons authority is a single point of failure. If the daemon crashes, the Leviathan falls. No new trust is established, no conflicts detected, no salvage occurs. Agents with cached Harbor Cards continue working, but the commons enters a state of nature until the daemon restarts. This is mitigated by automatic restart (launchd on macOS) but not formally bounded.

Collateral does not prevent harm—it prices it. A principal willing to burn money to sabotage a competitor will post a bond, cause damage, and accept the loss. The bond makes sabotage expensive, not impossible. The defense against existential threats is not internal governance but *admission control*: who is allowed to participate in the commons at all. This decision is irreducibly political and lies outside the formal model.

Single-node scope. The formal model assumes all agents share a single machine and a single SQLite database. Distributed multi-node settings require consensus primitives (Paxos [11], Raft [12]) that are unnecessary for local development and would introduce complexity disproportionate to the threat model.

Recovery fidelity depends on LLM capability. Social recovery works because successor agents can interpret natural-language evidence trails. This capability is not formally guaranteed—it depends on the successor's LLM. A formal model of "given this evidence trail, will the successor complete the original intent?" requires characterizing LLM reasoning, which is an open problem.

Bond pricing is unsolved. We identify the pricing function π as the critical open problem. Without an adequate π, bonds are either too low (cheap sabotage) or too high (priced-out legitimate agents). This is a mechanism design problem, not a systems problem, and requires expertise we explicitly flag as needed.

11 Conclusion

We have argued that multi-agent collaboration at scale requires a commons authority—a pre-transactional trust infrastructure—rather than peer-to-peer trust negotiation. The authority provides three layers of guarantee:

1. **Structural prevention**: Capability-attenuated tokens make scope escalation physically impossible.
2. **Immutable attribution**: Merkle-chained evidence trails make anonymous harm impossible.
3. **Economic alignment**: Collateralized work contracts make harm expensive, transforming moral questions into economic ones.

The advisory design of the resource claim system is not an engineering shortcut—it is the correct resolution of Sen's impossibility result for systems where agents have private knowledge about their own needs. The commons authority provides information (the conflict graph), not allocation decisions.

The central open problem is bond pricing: designing a function π that makes defection too expensive for rational adversaries without pricing legitimate agents out of the commons. This is a mechanism design problem, and we invite collaboration with economists to solve it.

The infrastructure is running. The formal model is specified. The economic layer awaits its market designer.

References

[1] Thomas Hobbes. *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiasticall and Civill*. Andrew Crooke, London, 1651.

[2] John Locke. *Two Treatises of Government*. Awnsham Churchill, London, 1689.

[3] Amartya Sen. The Impossibility of a Paretian Liberal. *Journal of Political Economy*, 78(1):152–157, 1970. `https://doi.org/10.1086/259614`

[4] Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987.

[5] Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[6] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[7] Anand S. Rao and Michael P. Georgeff. Modeling Rational Agents within a BDI-Architecture. In *International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991.

[8] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

[9] Erich Owens. The Anchor Protocol: A Formally Verified Control Plane for Local Agent Swarms. Technical White Paper, Port Daddy Engineering Team, March 2026.

[10] Leslie Lamport. *Specifying Systems: The TLA$^+$ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[11] Leslie Lamport. The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.

[12] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–320, 2014.

[13] Jack B. Dennis and Earl C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.

[14] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.

[15] William Vickrey. Counterspeculation, Auctions, and Competitive Sealed Tenders. *The Journal of Finance*, 16(1):8–37, 1961.

[16] Roger B. Myerson. Optimal Auction Design. *Mathematics of Operations Research*, 6(1):58–73, 1981.

[17] Jonathan Hickman. *House of X / Powers of X*. Marvel Comics, 2019.

[18] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016.